PROJECT REPORT

EXPERIMENT - 03

PROJECT TITLE

Smart Elevator Automation and Human Detection System

Submitted by

MUTHU KRISHNA P - 24MER033

PRIYANTH C S - 24MER044

SRI RAMAN G - 24MEL068

PRADEEP P - 24MER041

Smart Elevator Automation and Human Detection System

TITEL

The **Smart Elevator Automation and Human Detection System** is a sensor-based automated elevator designed to improve **safety, convenience, and energy efficiency**. The system uses human detection sensors to identify passengers and automatically activates the elevator.

1. Project Overview

The **Smart Elevator Automation and Human Detection System** is an automated elevator model designed to improve safety, convenience, and energy efficiency. The system uses human detection sensors to identify passengers near the elevator and automatically activates the elevator system. A microcontroller such as an **ESP8266 or Arduino** processes the sensor inputs and controls the elevator motor, floor selection, indicators, and buzzer.

The prototype demonstrates how automation and sensor-based control can be applied to modern elevator systems.

2. Problem Statement

Conventional elevator systems require manual interaction and may consume energy even when there are no passengers waiting. Human presence is not always considered during elevator operation. In addition, improper floor positioning, door operation, and passenger detection can create safety concerns.

Therefore, there is a need for a smart elevator system that can automatically detect humans, respond to floor requests, control elevator movement, and improve operational safety.

3. Proposed Solution

The proposed system uses a **human detection sensor** to detect passengers waiting for the elevator. A microcontroller receives the sensor information and controls the elevator motor through a motor driver.

The system includes:

- Human presence detection
- Floor selection buttons

- Automatic elevator movement
- Floor position detection
- Automatic door operation
- Floor display
- Buzzer alert
- Safety detection
- Energy-efficient operation

The system can be developed as a miniature working prototype for demonstrating smart elevator automation.

4. System Architecture

The overall system consists of the following blocks:

Human Detection Sensor → Microcontroller → Motor Driver → Elevator Motor

Additional inputs and outputs are connected to the controller:

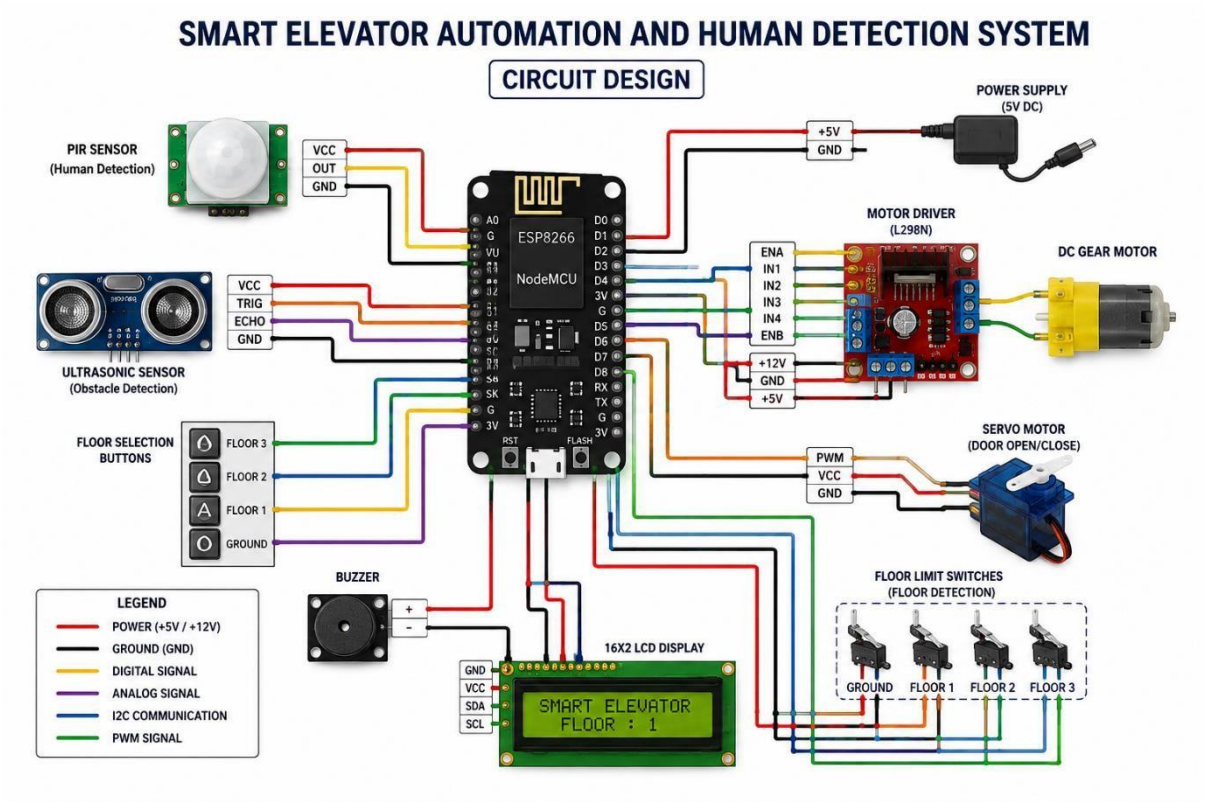
Floor Buttons → Microcontroller → Floor Display Floor Sensors → Microcontroller → Motor Control Door Sensor → Microcontroller → Door Motor/Servo Microcontroller → Buzzer + LED Indicators

The microcontroller acts as the central control unit. It processes information from the sensors and generates appropriate control signals for the elevator motor and other output devices.

5. Circuit Design

The **ESP8266/Arduino** is connected to all sensors and actuators. The human detection sensor provides an input signal to the controller. Floor-selection buttons are also connected to the controller.

The controller sends signals to the **motor driver**, which operates the DC geared motor. Limit switches or position sensors are used to identify the elevator's floor position. A servo motor can be used to demonstrate automatic door opening and close



6. Circuit Table

Component	Connection	Purpose
ESP8266/Arduino	Main controller	Controls complete system
IR/PIR Sensor	Digital input	Detects human presence
Ultrasonic Sensor	Digital input	Detects distance/obstacles
Push Buttons	Digital inputs	Select floors
Motor Driver	Digital/PWM outputs	Controls elevator motor
DC Gear Motor	Motor driver output	Moves elevator cabin
Servo Motor	PWM output	Controls elevator door
Limit Switches	Digital inputs	Detect floor position
LCD/OLED	I^2C	Displays floor/status
Buzzer	Digital output	Provides alerts
LEDs	Digital outputs	Indicates elevator status
Power Supply	VCC/GND	Supplies required power

7. Algorithm

- If no person is detected, keep the elevator in standby mode.
- If a person is detected, activate the elevator system.
- Check the selected floor.
- Compare the current floor with the requested floor.
- If the requested floor is above the current floor, rotate the motor in the upward direction.
- If the requested floor is below the current floor, rotate the motor in the downward direction.
- Monitor the floor-position sensor continuously.
- Stop the motor when the required floor is reached.
- Open the elevator door.
- Wait for the passenger to enter/exit.
- Close the door.
- Activate the buzzer/indicator as required.
- Return to standby mode

8. Coding

The controller program continuously reads the human detection sensor, floor-selection buttons, and position sensors. Based on these inputs, it controls the elevator motor, door servo, LEDs, and buzzer.

```
#include <ESP8266WiFi.h>

#include <Adafruit_MQTT.h>

#include <Adafruit_MQTT_Client.h>

#include <Servo.h>

//
============================================
=======

// WIFI
```

```
//
============================================= =======


const char* WIFI_SSID = "Pradeep"; const char*

WIFI_PASSWORD = "12345678";


//
=============================================
=======

// ADAFRUIT IO

//
============================================= =======


#define AIO_SERVER       "io.adafruit.com"

#define AIO_SERVERPORT   1883


#define AIO_USERNAME     "muthumkt"

#define AIO_KEY
"aio_Dpts45gXpVu4he3HHcwUXVBgOuKb"


//
=============================================
=======

// PIN CONNECTIONS

//
============================================= =======


#define PIR_PIN       D5

#define SERVO_PIN     D4

#define LED_PIN       D1
```

```
#define BUZZER_PIN    D3

#define BUTTON_1      D6

#define BUTTON_2      D7


//
================================================
=======

// SERVO
//
================================================ =======


Servo elevatorServo;


int CLOSED_POSITION = 0; int

OPEN_POSITION   = 90;


//
================================================
=======

// WIFI + MQTT
//
================================================ =======


WiFiClient client;


Adafruit_MQTT_Client mqtt(

  &client,

  AIO_SERVER,

  AIO_SERVERPORT,
```

```
  AIO_USERNAME,
  AIO_KEY
);

//
================================================
=======
// ADAFRUIT IO FEEDS
//
================================================ =======

Adafruit_MQTT_Publish humanFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/human-detected"
  );

Adafruit_MQTT_Publish floorFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/floor"
  );

Adafruit_MQTT_Publish doorFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/door-status"
```

```
  );

Adafruit_MQTT_Publish buzzerFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/buzzer"
  );

//
================================================
=======
// VARIABLES
//
================================================ =======

int humanDetected = 0; int
previousHumanDetected = 0;

int selectedFloor =
0; int doorStatus =
0; unsigned long
lastSendTime = 0;

const unsigned long SEND_INTERVAL = 15000;

//
================================================
=======
```

```
// BUZZER
//
============================================== =======

void beep(int timeMs)
{
  digitalWrite(BUZZER_PIN, HIGH);
delay(timeMs);   digitalWrite(BUZZER_PIN,
LOW);
}

//
==============================================
=======
// WIFI CONNECTION
//
============================================== =======

void connectWiFi()
{
  Serial.println();
  Serial.println("==============================");
  Serial.println("CONNECTING TO WIFI");
  Serial.println("==============================");

  Serial.print("WiFi Name: ");
  Serial.println(WIFI_SSID);
```

```
WiFi.mode(WIFI_STA);

WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

int attempts = 0;

while (WiFi.status() != WL_CONNECTED && attempts < 30)
{
  delay(500);

  Serial.print(".");

  attempts++;
}

Serial.println();

if (WiFi.status() == WL_CONNECTED)
{
  Serial.println("WIFI CONNECTED!");

  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());

  Serial.print("Signal Strength: ");
  Serial.print(WiFi.RSSI());
```

```
    Serial.println(" dBm");
  }
  else
  {
    Serial.println("WIFI CONNECTION FAILED!");

    Serial.println("Check:");
    Serial.println("1. WiFi name");
    Serial.println("2. WiFi password");
    Serial.println("3. Use 2.4 GHz WiFi");
  }
}

//
================================================
=======
// ADAFRUIT IO CONNECTION
//
================================================ =======

void connectMQTT()
{
  if (WiFi.status() != WL_CONNECTED)

{

r
e
```

```
t
u
r
n
;




}



  if (mqtt.connected())

{

r
e
t
u
r
n
;

  }

  Serial.println();
  Serial.println("Connecting to Adafruit IO...");
```

```
int8_t result;

for (int attempt = 1; attempt <= 5; attempt++)
{
  Serial.print("Attempt ");
  Serial.print(attempt);
  Serial.println("...");

  result = mqtt.connect();

  if (result == 0)
  {
    Serial.println("ADAfruit IO CONNECTED!");
    return;
  }

  Serial.print("Connection failed: ");
  Serial.println(mqtt.connectErrorString(result));

  mqtt.disconnect();

  delay(3000);
}

Serial.println("Could not connect to Adafruit IO.");
```

```
}

//
==========================================
=======
// SEND DATA
//
========================================== =======

void sendData()
{
  if (WiFi.status() != WL_CONNECTED)
  {
    Serial.println("WiFi not connected. Cannot send data.");     return;
  }

  if (!mqtt.connected())
  {
    connectMQTT();
  }

  if (!mqtt.connected())
  {
    Serial.println("Adafruit IO not connected.");     return;
  }

   Serial.println(); Serial.println("============================");
```

```
  Serial.println("SENDING DATA");
  Serial.println("==============================");

  // Human detection   if
(humanFeed.publish(humanDetected))
  {
    Serial.print("Human: ");
    Serial.println(humanDetected);
  }
  else
  {
    Serial.println("Human data FAILED");
  }

  // Floor   if
(floorFeed.publish(selectedFloor))
  {
    Serial.print("Floor: ");
    Serial.println(selectedFloor);
   }
   else
   {
    Serial.println("Floor data FAILED");
  }
```

```
  // Door   if
(doorFeed.publish(doorStatus))
  {
    Serial.print("Door: ");
    Serial.println(doorStatus);
  }
  else
  {
    Serial.println("Door data FAILED");
  }

  // Buzzer   if
(buzzerFeed.publish(0))
  {
    Serial.println("Buzzer data sent");
   }
   else
   {
    Serial.println("Buzzer data FAILED");
  }

  Serial.println("==============================");
}

//
==========================================
=======
```

```
// SETUP
//
============================================== =======


void setup()

{

  Serial.begin(115200);


  delay(2000);



   Serial.println();
   Serial.println();

   Serial.println("#################################");

  Serial.println("# SMART ELEVATOR SYSTEM");

  Serial.println("# ESP8266 NODEMCU");

  Serial.println("#################################");


  //
============================================
=====

  // PIN SETUP
  //
============================================== =====


  pinMode(PIR_PIN, INPUT);


  pinMode(LED_PIN, OUTPUT);
```

```
  pinMode(BUZZER_PIN, OUTPUT);

  pinMode(BUTTON_1, INPUT_PULLUP);

   pinMode(BUTTON_2, INPUT_PULLUP);

   // Initial output states   digitalWrite(LED_PIN,
LOW);

  digitalWrite(BUZZER_PIN, LOW);

  //
==========================================
=====
  // SERVO
  //
========================================== =====

  Serial.println("Starting Servo...");

  elevatorServo.attach(SERVO_PIN);

  elevatorServo.write(CLOSED_POSITION);

  delay(500);

   Serial.println("Servo OK");
```

```
  //
==========================================
=====

  // WIFI
  //
========================================== =====


  connectWiFi();


  //
==========================================
=====

  // ADAFRUIT IO
  //
========================================== =====


  if (WiFi.status() == WL_CONNECTED)

  {

    connectMQTT();

  }

  //
==========================================
=====

  // READY
  //
========================================== =====


  Serial.println();

  Serial.println("################################");

  Serial.println("# SYSTEM READY");
```

```
  Serial.println("#################################");
  Serial.println();
}

//
================================================
=======
// LOOP
//
================================================ =======

void loop()
{
  //
================================================
=====
  // CHECK WIFI
  //
================================================ =====

  if (WiFi.status() != WL_CONNECTED)
  {
    Serial.println("WiFi disconnected!");     connectWiFi();
  }

  //
================================================
=====
  // CHECK ADAFRUIT IO
  //
================================================ =====
```

```
  if (WiFi.status() == WL_CONNECTED)
  {
    if (!mqtt.connected())
    {
      connectMQTT();
    }
  }

  //
==============================================
=====
  // PIR SENSOR
  //
============================================== =====

  humanDetected = digitalRead(PIR_PIN);

  if (humanDetected == HIGH)
  {
    // Human detected     digitalWrite(LED_PIN,
HIGH);

    elevatorServo.write(OPEN_POSITION);

    doorStatus = 1;

    if (previousHumanDetected == 0)
```

```
    {
      Serial.println(">>> HUMAN DETECTED");

      beep(200);
    }
  }
  else
  {
    // No human     digitalWrite(LED_PIN,
LOW);

    elevatorServo.write(CLOSED_POSITION);

    doorStatus = 0;

    if (previousHumanDetected == 1)
    {
      Serial.println(">>> NO HUMAN");
    }
   }

  previousHumanDetected = humanDetected;

  //
==============================================
=====
  // BUTTON 1
```

```
 //
============================================== =====

 if (digitalRead(BUTTON_1) == LOW)
 {
   selectedFloor = 1;

   Serial.println(">>> FLOOR 1 SELECTED");

   beep(150);

   delay(500);
 }
  //
============================================== =====
 // BUTTON 2
 //
============================================== =====

 if (digitalRead(BUTTON_2) == LOW)
 {
   selectedFloor = 2;

   Serial.println(">>> FLOOR 2 SELECTED");

   beep(150);
```

```
    delay(500);

  }


  //
==========================================
=====

  // SEND TO ADAFRUIT IO EVERY 15 SECONDS

   //
========================================== =====


  if (millis() - lastSendTime >= SEND_INTERVAL)

  {

    lastSendTime = millis();


    sendData();

  }


  delay(100);

}
```

9. System Working

When the system is switched ON, the microcontroller initializes all connected components. The human detection sensor continuously monitors the area around the elevator.

When a person approaches the elevator, the sensor detects the person's presence and sends a signal to the controller. The system then becomes active and waits for the required floor to be selected.

After the floor is selected, the controller determines whether the elevator needs to move upward or downward. The motor driver operates the DC geared motor in the required direction.

Position sensors continuously monitor the elevator cabin. When the cabin reaches the selected floor, the controller stops the motor and activates the door mechanism.

The door opens automatically, allowing the passenger to enter or exit. After a predetermined time, the door closes and the system returns to standby mode.

10. Bill of Materials

S.No	Component	Quantity
1	ESP8266 / Arduino	1
2	IR/PIR Human Detection Sensor	1
3	Ultrasonic Sensor	1
4	DC Geared Motor	1
5	Motor Driver	1
6	Servo Motor	1
7	Push Buttons	3–5
8	Limit Switches	2–4
9	LCD/OLED Display	1
10	LEDs	3–5
11	Buzzer	1
12	Power Supply	1
13	Connecting Wires	As required
14	Elevator Model/Frame	1
15	Breadboard/PCB	1

11. Prototype Implementation

A miniature elevator cabin is constructed using lightweight materials such as acrylic sheet, cardboard, or plywood. The cabin is connected to a DC geared motor using a pulley, belt, thread, or rack-and-pinion mechanism.

The controller is installed on the base of the prototype. Human detection sensors are positioned near the elevator entrance. Floor sensors or limit switches are installed at different levels to identify the elevator position.

Push buttons are provided for floor selection. LEDs and an LCD/OLED display indicate the elevator status. A servo motor can be installed to demonstrate automatic door opening and closing.

The complete prototype is tested under different conditions to verify human detection, floor selection, motor movement, floor positioning, and door operation.

12. Results & Performance Analysis

The prototype successfully demonstrates automated elevator operation based on human detection and floor selection.

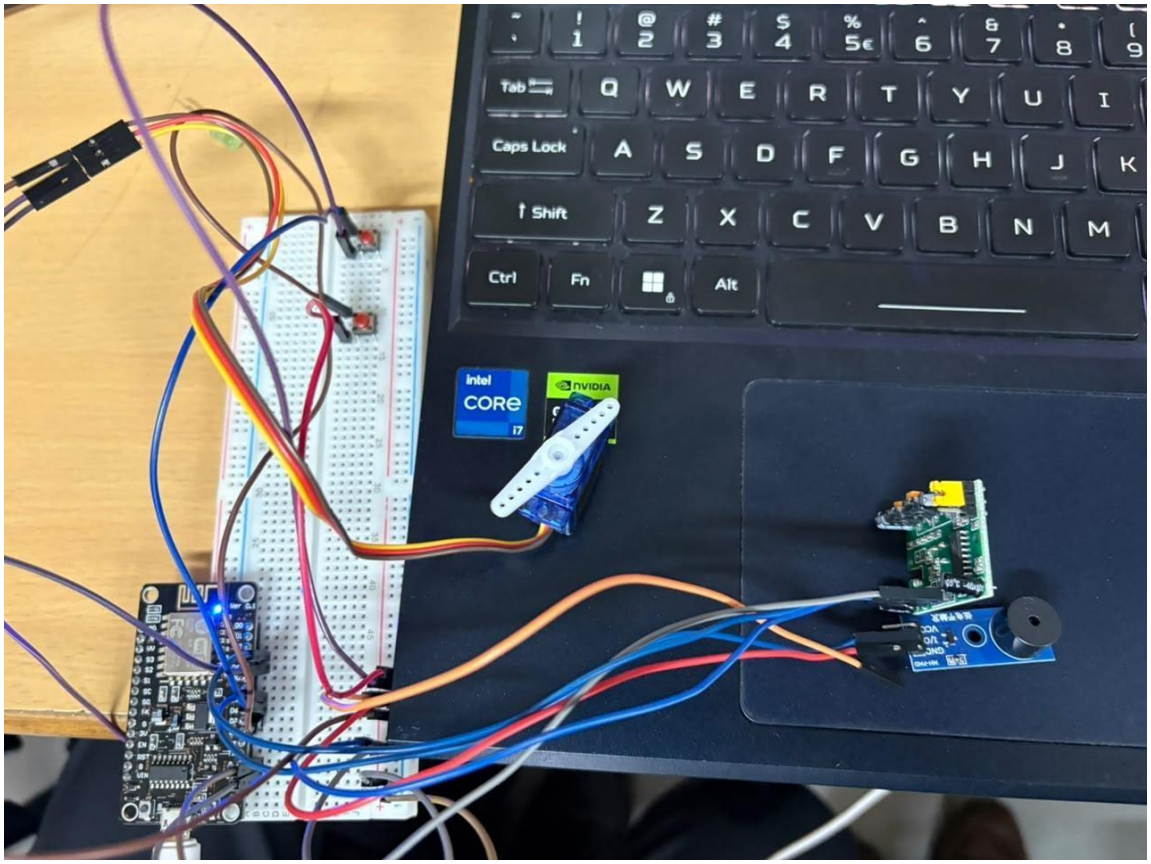
Parameter	Expected Result
Human Detection	Successfully detects approaching person
Floor Selection	Responds to selected floor

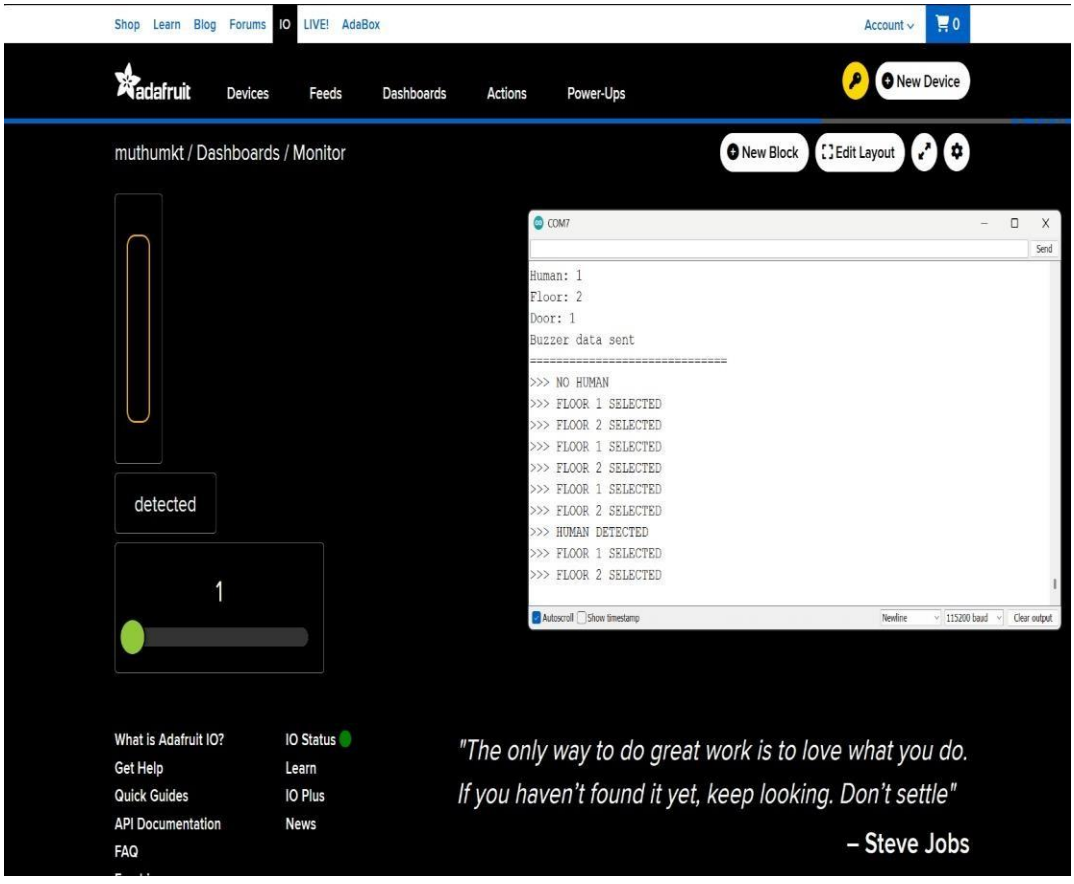
Parameter	**Expected Result**
Elevator Movement	Moves in required direction
Floor Positioning	Stops at selected floor
Door Operation	Automatically opens/closes
Floor Indication	Displays current floor
Alert System	Buzzer provides warning
Safety Detection	Detects obstacles/presence
Automation	Reduces manual operation
Energy Efficiency	Elevator remains in standby when unused

Overall Result

The **Smart Elevator Automation and Human Detection System** provides a practical demonstration of sensor-based automation. The prototype integrates **human detection, motor control, floor sensing, automatic door control, and status indication** into a single system. The project can be further improved by adding IoT monitoring, voice commands, emergency communication, overload detection, and mobile-app control.

OUTPUT:




Shop Learn Blog Forums IO LIVE! AdaBox
Account 0
adafruit
Devices Feeds Dashboards Actions Power-Ups
New Device
muthumkt / Dashboards / Monitor
New Block
Edit Layout
COM7
Send
Human: 1
Floor: 2
Door: 1
Buzzer data sent
==========
>>> NO HUMAN
>>> FLOOR 1 SELECTED
>>> FLOOR 2 SELECTED
>>> FLOOR 1 SELECTED
>>> FLOOR 2 SELECTED
>>> FLOOR 1 SELECTED
>>> FLOOR 2 SELECTED
>>> HUMAN DETECTED
>>> FLOOR 1 SELECTED
>>> FLOOR 2 SELECTED
Autoscroll
Show timestamp
Newline
115200 baud
Clear output
detected
1
What is Adafruit IO?
Get Help
Quick Guides
API Documentation
FAQ
IO Status
Learn
IO Plus
News
"The only way to do great work is to love what you do.
If you haven't found it yet, keep looking. Don't settle"
– Steve Jobs